Homework 07

Use either R or Python.

Question 1: LDA

Let's start by estimating the components of LDA. From the lecture notes, for each target point $\mathbf{x}_0$, we find the class label k with the largest score

$$\mathbf{x}_0^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k.$$

Thus, we need to estimate:

- the prior probabilities π_k;
- the mean vector (centroid) μ_k for each class; and
- the pooled covariance matrix Σ.

Use saheart.csv, the `SAheart` data from `ElemStatLearn`. The two classes are defined by `chd`, the coronary heart disease indicator. Use all nine other variables as numerical predictors, coding `famhist` as 1 for `Absent` and 2 for `Present`. The following code constructs an $n \times 9$ predictor matrix $\mathbf{X}$ and an n-entry response vector $\mathbf{y}$, where n is the number of observations.

R

```r
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd
```

Python

```python
import pandas as pd

heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy()
y = heart["chd"].to_numpy()
```

a. Calculate the prior probabilities, class means, and pooled covariance matrix from these data. Use the pooled covariance denominator $n - K$ from the lecture, where $K = 2$ is the number of classes.

```r
heart <- read.csv(
  "/Users/jessicacudzich/Desktop/stat432-fall2026/homework/week-07/data/saheart.csv"
)

heart$famhist <- factor(
  heart$famhist,
  levels = c("Absent", "Present")
)

X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd

classes <- sort(unique(y))
n <- nrow(X)
K <- length(classes)

priors <- sapply(classes, function(k) mean(y == k))
names(priors) <- classes

mu <- t(sapply(classes, function(k) {
  colMeans(X[y == k, , drop = FALSE])
}))
rownames(mu) <- classes

centered <- X - mu[match(y, classes), ]

Sigma <- crossprod(centered) / (n - K)

table(y)
```

```
y
  0   1
302 160
```

```
round(priors, 6)
```

```
      0       1
0.65368 0.34632
```

```
round(mu, 6)
```

```
       sbp  tobacco      ldl adiposity  famhist    typea  obesity  alcohol
0 135.4603 2.634735 4.344238  23.96911 1.317881 52.36755 25.73745 15.93136
1 143.7375 5.524875 5.487938  28.12025 1.600000 54.49375 26.62294 19.14525
       age
0 38.85430
1 50.29375
```

```
round(Sigma, 6)
```

```
                sbp   tobacco       ldl adiposity  famhist      typea
sbp       405.434779 14.585188  4.581248 49.164358 0.336988 -15.587722
tobacco    14.585188 19.242556  0.763203  7.538109 0.015821  -2.057306
ldl         4.581248  0.763203  4.000581  6.032697 0.091850   0.344592
adiposity  49.164358  7.538109  6.032697 56.752893 0.432808  -5.309566
famhist     0.336988  0.015821  0.091850  0.432808 0.225834   0.081120
typea     -15.587722 -2.057306  0.344592 -5.309566 0.081120  95.565643
obesity    18.938843  1.833451  2.660052 22.707885 0.184028   2.640067
alcohol    64.400352 20.517045 -2.532907 16.119025 0.768469   7.960032
age        95.133889 22.765411  6.478920 60.508672 0.997363 -20.278335
            obesity    alcohol        age
sbp       18.938843  64.400352  95.133889
tobacco    1.833451  20.517045  22.765411
ldl        2.660052  -2.532907   6.478920
adiposity 22.707885  16.119025  60.508672
famhist    0.184028   0.768469   0.997363
typea      2.640067   7.960032 -20.278335
obesity   17.615424   4.689356  15.696999
alcohol    4.689356 598.276613  27.885941
age       15.696999  27.885941 184.132137
```

There are (n=462) observations: 302 in class 0 and 160 in class 1. Thus, the estimated prior probabilities are 0.653680 and 0.346320.

b. Use your estimated values to predict the label of each observation in the training data. These are in-sample fitted labels. Provide the confusion matrix, with observed classes in rows and predicted classes in columns. Check that the dimensions of your matrices and vectors match. Calculate the scores yourself for this part.

```
weights <- solve(Sigma, t(mu))

intercepts <- -0.5 * rowSums(mu * t(weights)) + log(priors)

scores <- sweep(X %*% weights, 2, intercepts, FUN = "+")
colnames(scores) <- classes

pred_manual <- classes[max.col(scores, ties.method = "first")]

list(
  X = dim(X),
  mu = dim(mu),
  Sigma = dim(Sigma),
  weights = dim(weights),
  intercepts = length(intercepts),
  scores = dim(scores),
  predictions = length(pred_manual)
)
```

```
$X
[1] 462   9

$mu
[1] 2 9

$Sigma
[1] 9 9

$weights
[1] 9 2

$intercepts
[1] 2
```

```
$scores
[1] 462   2

$predictions
[1] 462
```

```r
cm_manual <- table(
  Observed = factor(y, levels = classes),
  Predicted = factor(pred_manual, levels = classes)
)

cm_manual
```

```
        Predicted
Observed   0   1
       0 258  44
       1  73  87
```

```r
mean(pred_manual == y)
```

```
[1] 0.7467532
```

The model correctly classifies (258+87=345) observations. Its training accuracy is (345/462=0.746753), or approximately 74.68%. The training misclassification rate is approximately 25.32%.

c. Perform the same LDA analysis using `MASS::lda` in R or `LinearDiscriminantAnalysis(solver="svd")` in Python, using the same numerical predictors and empirical class priors. Provide the confusion matrix. Do the results match?

```r
fit_lda <- MASS::lda(
  x = X,
  grouping = factor(y, levels = classes),
  prior = unname(priors)
)

pred_lda <- predict(fit_lda, newdata = X)$class

cm_lda <- table(
  Observed = factor(y, levels = classes),
  Predicted = factor(pred_lda, levels = classes)
```

```
)

cm_lda
```

```
        Predicted
Observed   0   1
       0 258  44
       1  73  87
```

```
all(as.character(pred_lda) == as.character(pred_manual))
```

```
[1] TRUE
```

Using `MASS::lda` with the same nine numerical predictors and empirical class priors gives the following confusion matrix:

Observed class	Predicted 0	Predicted 1
0	258	44
1	73	87

The results match the manual LDA calculation exactly. All 462 predicted labels agree, and both methods have a training accuracy of approximately 74.68%.

Question 2: QDA and marginal screening

In the lecture, ordinary QDA could not be fitted to the handwritten digit sample because its within-class covariance matrices were singular. Let's consider selecting fewer variables and see whether this helps. For this question, use the full zip-train.csv.gz as the training data and zip-test.csv.gz as the test data, with all ten digit classes. These are the original `zip.train` and `zip.test` data from `ElemStatLearn`. Each file has no header: the first column is the digit label and the remaining 256 columns are pixel values.

Instead of using all 256 predictors, select the 40 with the largest marginal variances in the training data. Here, marginal variance means the variance of one pixel across all training observations. Use the original pixel values without standardizing them, and retain the same selected predictors in the test data.

Use `MASS::qda` in R or `QuadraticDiscriminantAnalysis(tol=1e-8)` in Python.

a. Fit QDA using these 40 predictors and report the test confusion matrix, with observed classes in rows and predicted classes in columns. Also report the test misclassification rate.

```
library(MASS)

train <- read.csv(gzfile("data/zip-train.csv.gz"), header = FALSE)
test <- read.csv(gzfile("data/zip-test.csv.gz"), header = FALSE)

X_train <- as.matrix(train[, -1])
X_test <- as.matrix(test[, -1])

y_train <- factor(train[, 1], levels = 0:9)
y_test <- factor(test[, 1], levels = 0:9)

pixel_variances <- apply(X_train, 2, var)
ranked_pixels <- order(pixel_variances, decreasing = TRUE)
selected_40 <- ranked_pixels[1:40]

X_train_40 <- X_train[, selected_40, drop = FALSE]
X_test_40 <- X_test[, selected_40, drop = FALSE]

fit_40 <- qda(x = X_train_40, grouping = y_train)

pred_40 <- predict(fit_40, newdata = X_test_40)$class

cm_40 <- table(
  Observed = y_test,
  Predicted = factor(pred_40, levels = 0:9)
)

error_40 <- mean(pred_40 != y_test)

cm_40
```

```
        Predicted
Observed   0   1   2   3   4   5   6   7   8   9
       0 319   0  24   6   2   2   1   0   5   0
       1   0 238   5   0  11   0   7   0   3   0
       2   8   0 168   8   8   3   0   0   3   0
       3   5   0   4 128   0  14   0   0  15   0
       4   0   2  13   1 150   1   0   1   7  25
```

```
  5  12   0   3   5   1 131   0   0   6   2
  6   0   0   8   0   1   8 148   0   5   0
  7   1   0   6   1   6   0   0 124   2   7
  8   1   0  10  12   3   2   0   0 137   1
  9   0   0   1   2   7   1   1   4   9 152
```

```
error_40
```

```
[1] 0.1554559
```

```
100 * error_40
```

```
[1] 15.54559
```

I selected the 40 pixels with the largest marginal variances in the training data. Each variance was calculated across all training observations, without separating the digit classes. I retained the same selected pixels in the test data and used their original values without standardization.

I fitted QDA using these 40 predictors and empirical class priors. The output above shows the test confusion matrix, with observed classes in rows and predicted classes in columns. The test misclassification rate was 0.1555, or 15.55%.

b. Does the method work? Why do you think it works or does not work?

```
ranks_40 <- sapply(levels(y_train), function(k) {
  class_data <- X_train_40[y_train == k, , drop = FALSE]
  qr(cov(class_data))$rank
})

data.frame(
  Digit = levels(y_train),
  Covariance_rank = unname(ranks_40)
)
```

```
  Digit Covariance_rank
1     0              40
2     1              40
3     2              40
4     3              40
5     4              40
```

```
6       5               40
7       6               40
8       7               40
9       8               40
10      9               40
```

The method works computationally because QDA can be fitted using the selected 40 predictors. The within-class covariance matrices have full rank, so they can be inverted.

Reducing the number of predictors reduces the number of covariance parameters that must be estimated for each digit class. Selecting pixels with large marginal variances also favors pixels that vary across images and removes many pixels with little variation. However, large overall variance does not necessarily mean that a pixel helps distinguish digit classes, and reducing the number of predictors does not always guarantee nonsingular within-class covariance matrices.

c. Repeat the analysis using the 10 predictors with the largest marginal variances in the training data. How does the test misclassification rate compare with the 40-predictor version? Why do you think this happens?

```r
selected_10 <- ranked_pixels[1:10]

X_train_10 <- X_train[, selected_10, drop = FALSE]
X_test_10 <- X_test[, selected_10, drop = FALSE]

fit_10 <- qda(x = X_train_10, grouping = y_train)

pred_10 <- predict(fit_10, newdata = X_test_10)$class

cm_10 <- table(
  Observed = y_test,
  Predicted = factor(pred_10, levels = 0:9)
)

error_10 <- mean(pred_10 != y_test)

cm_10
```

```
         Predicted
Observed   0   1   2   3   4   5   6   7   8   9
       0 298   0  11   1   1  16  28   4   0   0
       1   0 241  10   0   3   1   1   0   2   6
```

```
   2  19   2  95  14  14   3  22  21   5   3
   3   5   0   6 103   3  42   0   0   7   0
   4   1   5  14   0  95   2   4  15  17  47
   5  15   0   6  25   4  96   4   2   8   0
   6  41   0  15   4   0  18  78   7   5   2
   7   4   1   6   0   6   0   1 122   0   7
   8   1   2  11  36  12  11   4   1  74  14
   9   0   2   9   2  20   0   0  16  10 118
```

```
data.frame(
  Predictors = c(40, 10),
  Test_misclassification_rate = c(error_40, error_10)
)
```

```
  Predictors Test_misclassification_rate
1         40                   0.1554559
2         10                   0.3423019
```

```
error_10 - error_40
```

```
[1] 0.186846
```

The test misclassification rate was 34.23% for the 10-predictor model, compared with 15.55% for the 40-predictor model. Thus, the 10-predictor model had a higher test misclassification rate.

Using fewer predictors involves a tradeoff. With 10 predictors, QDA estimates fewer covariance parameters, which can make estimation more stable. However, it also discards pixels that may contain useful information for distinguishing digits. A higher error rate would suggest that the loss of useful information outweighs the benefit of simpler covariance estimation; a lower error rate would suggest that reducing estimation variability is beneficial.

Question 3: Compare LDA and QDA

In this simulation, we will study how the relative performance of LDA and QDA changes with the true covariance structure. The parameter ρ controls the distribution from which we generate the data; it is not a tuning parameter of LDA or QDA.

Consider a classification problem with two equally likely classes, labeled 1 and 2, and $p = 10$ covariates. The first two covariates have the class-conditional distributions

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \mid Y = 1 \sim N_2 \left(\begin{pmatrix} -1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \right),$$

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \mid Y = 2 \sim N_2 \left(\begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix} \right).$$

In both classes, the remaining eight covariates are independent standard normal variables, independent of the first two. Consider

$$\rho \in \{0, 0.2, 0.4, 0.6, 0.8, 0.9\}.$$

For each value of ρ, generate 100 training observations and 1,000 independent test observations from each class. Fit ordinary LDA and QDA using all 10 covariates and equal class priors. Estimate the class means from the training data, with one pooled covariance matrix for LDA and one full covariance matrix per class for QDA. Set a random seed and report it.

a. Generate one dataset for each value of ρ. Fit both methods and calculate their test misclassification rates. Within each setting, use the same training and test observations for the two methods.

```r
library(MASS)

set.seed(432)

rho_values <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)

generate_data <- function(n_per_class, rho) {

  make_class <- function(label) {
    z <- matrix(
      rnorm(n_per_class * 10),
      nrow = n_per_class,
      ncol = 10
    )

    correlation <- if (label == 1) rho else -rho

    x <- z
    x[, 1] <- z[, 1] + if (label == 1) -1 else 1
    x[, 2] <- correlation * z[, 1] +
      sqrt(1 - correlation^2) * z[, 2]
```

```
    x
  }

  list(
    X = rbind(make_class(1), make_class(2)),
    y = factor(rep(1:2, each = n_per_class))
  )
}

run_experiment <- function(rho) {
  train <- generate_data(100, rho)
  test <- generate_data(1000, rho)

  lda_fit <- lda(
    x = train$X,
    grouping = train$y,
    prior = c(0.5, 0.5)
  )

  qda_fit <- qda(
    x = train$X,
    grouping = train$y,
    prior = c(0.5, 0.5)
  )

  c(
    LDA = mean(predict(lda_fit, test$X)$class != test$y),
    QDA = mean(predict(qda_fit, test$X)$class != test$y)
  )
}

single_results <- data.frame(
  rho = rho_values,
  t(sapply(rho_values, run_experiment))
)

knitr::kable(single_results, digits = 4)
```

```
Warning: 'xfun::attr()' is deprecated.
Use 'xfun::attr2()' instead.
See help("Deprecated")
```

```
Warning: 'xfun::attr()' is deprecated.
Use 'xfun::attr2()' instead.
See help("Deprecated")
```

rho	LDA	QDA
0.0	0.1505	0.1695
0.2	0.1650	0.1890
0.4	0.1660	0.1920
0.6	0.1775	0.1760
0.8	0.1695	0.1355
0.9	0.1610	0.1110

In this simulation, LDA had lower test error at rho = 0, 0.2, and 0.4. QDA had slightly lower error at rho = 0.6 and more substantially lower error at rho = 0.8 and 0.9.

b. At each fixed value of ρ, repeat the experiment 50 times, generating new training and test observations in each repetition. Plot average test error against ρ in one figure, with one curve for LDA and one for QDA. These curves compare performance across different data-generating distributions.

```r
set.seed(433)

B <- 50

average_results <- do.call(
  rbind,
  lapply(rho_values, function(rho) {

    errors <- replicate(B, run_experiment(rho))

    data.frame(
      rho = rho,
      LDA = mean(errors["LDA", ]),
      QDA = mean(errors["QDA", ])
    )
  })
)

knitr::kable(average_results, digits = 5)
```

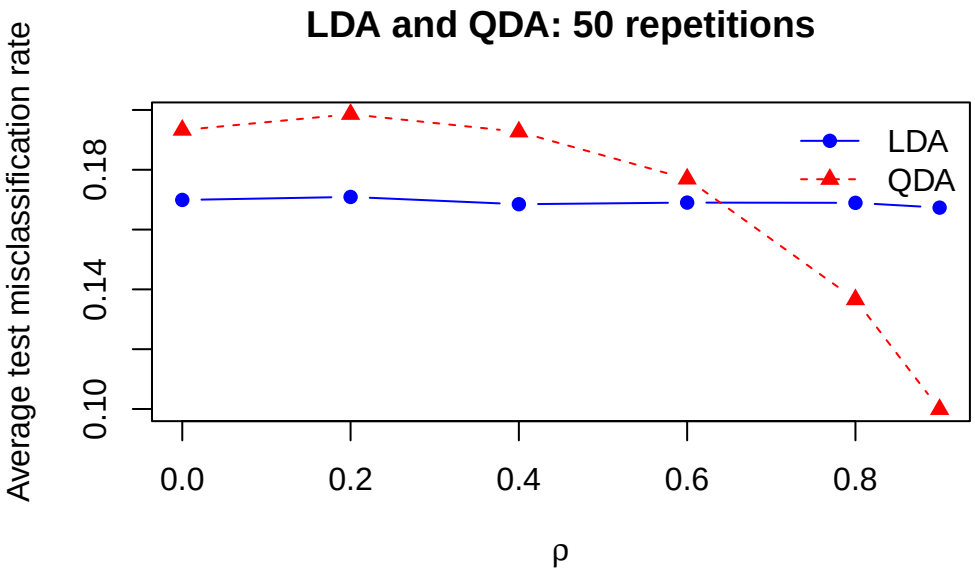
```
Warning: 'xfun::attr()' is deprecated.
Use 'xfun::attr2()' instead.
See help("Deprecated")

Warning: 'xfun::attr()' is deprecated.
Use 'xfun::attr2()' instead.
See help("Deprecated")
```

rho	LDA	QDA
0.0	0.16991	0.19327
0.2	0.17091	0.19857
0.4	0.16848	0.19269
0.6	0.16902	0.17695
0.8	0.16894	0.13659
0.9	0.16734	0.09988

```r
matplot(
  average_results$rho,
  as.matrix(average_results[, c("LDA", "QDA")]),
  type = "b",
  pch = c(16, 17),
  lty = c(1, 2),
  col = c("blue", "red"),
  xlab = expression(rho),
  ylab = "Average test misclassification rate",
  main = "LDA and QDA: 50 repetitions"
)

legend(
  "topright",
  legend = c("LDA", "QDA"),
  col = c("blue", "red"),
  pch = c(16, 17),
  lty = c(1, 2),
  bty = "n"
)
```



I used random seed 433 for the repeated simulation. At each fixed value of rho, I repeated the experiment 50 times, generating new training and test observations in every repetition. Within each repetition, LDA and QDA used the same observations.

LDA's average test error remained close to 17% across the settings. QDA had higher average error at small values of rho, but its error decreased substantially at large values. QDA outperformed LDA at rho = 0.8 and 0.9. These curves compare different data-generating distributions; rho is not a tuning parameter of either fitted method.

c. Compare the two methods at small and large values of ρ. How does changing the true covariance structure affect their relative performance? Explain the pattern using their covariance assumptions and the bias-variance trade-off. How can the eight noise covariates affect performance even though they contain no information about the class label?

When rho is zero, both classes have the same covariance matrix, so LDA's shared-covariance assumption is correct. LDA pools observations from both classes to estimate one covariance matrix, giving it lower estimation variance than QDA. QDA estimates a separate covariance matrix for each class, which introduces additional variability without a population-level advantage when the covariances are equal.

At small positive values of rho, the class covariance matrices differ only modestly. LDA's lower estimation variance can still outweigh the bias from assuming a common covariance matrix. This explains its lower average test error in these settings.

At large values of rho, the first two covariates have a strong positive correlation in class 1 and a strong negative correlation in class 2. QDA can use this difference through a quadratic decision boundary. LDA pools the two opposing correlations, which approximately cancel, and cannot fully use the class-specific covariance structure. QDA's reduction in bias then outweighs its greater estimation variance, leading to lower average test error.

The eight noise covariates contain no information about the class label, but their estimated means, variances, and correlations fluctuate in finite samples. These random differences can influence the fitted decision boundaries and hurt test performance. This is especially relevant for QDA: with 10 predictors, it estimates 55 distinct covariance entries per class, compared with 55 total for LDA's pooled covariance matrix.

Question 4: Create and use an LDA/QDA analysis skill

In Homework 5, you created a skill for formatting plots. Now create a reusable skill that guides an AI agent through an LDA/QDA analysis and produces a concise statistical report. You may work with AI to design and write the skill; you do not need to write every line yourself.

a. **Define the skill.** Write a single `SKILL.md` file no larger than **5 KB (5,000 bytes)**. Define the agent's behavior, analysis guidelines, and expected report structure. Keep the skill independent of this question: provide the dataset, response, and predictors separately when invoking it. The skill may guide the agent to explore lower-dimensional representations, such as principal component analysis (PCA), or to blend the class probabilities predicted by LDA and QDA. Keep all classifiers based on LDA, QDA, or their combination. Configure the skill to load **only when you explicitly invoke it**. Use your AI tool's invocation settings, or keep the file outside automatically loaded skill folders and supply it only in an explicit request.

Your skill must require a clearly readable analysis report of **at most one printed page**, containing a figure, a table, and numerical model-fitting results. You may specify the type and style of figures and tables you want the AI to produce. All instructions in the skill must be independent of the actual dataset, its variable names, and its observed results. The report should briefly describe the candidate models, the cross-validation procedure and results, the selected model, and its estimated prediction error. It should explain the main findings and any limitations.

b. **Apply the skill.** Explicitly invoke your skill to analyze the `SAheart` data from Question 1, using the same response and predictor coding. Ask the agent to seek the best predictive performance within the LDA/QDA methods allowed by your skill. Include ordinary LDA and QDA as comparisons; the extensions above are optional. Reserve a test set before choosing models. Use the same cross-validation folds within the training data to compare candidates and select any settings. Estimate any preprocessing, including an embedding, using only the training observations in each fold. Refit the selected model

and preprocessing on all training observations, then evaluate the fitted classifier once on the reserved test set. Use misclassification rate to compare performance, and report your random seed.

c. **Review and submit.** Check the agent's analysis and make any skill revisions using only training and cross-validation results. Append the final one-page analysis report and the complete contents of your `SKILL.md` to your homework report. State the file size and briefly explain how you enabled explicit invocation, including the prompt used to apply the skill. The one-page limit applies to the analysis report; the appended skill and this explanation are outside that limit.

```
file.info("/Users/jessicacudzich/Desktop/stat432-fall2026/homework/week-07/data/lda-qda-analy
```

```
[1] 2562
```

My `SKILL.md` file is 2562 bytes, below the 5,000-byte limit. I saved it outside automatically loaded skill folders so it would be used only when explicitly requested. I invoked it by supplying the file to the AI assistant and asking it to apply the skill to the SAheart dataset. The invocation prompt and complete skill contents are included below.

Your skill and analysis report will be reviewed and graded by other students through peer evaluation.